

FinNova Retail – Order Management Cloud Migration

L2 Technical Hands-On Assessment – Strategy, Architecture, Runbook and Cost

Ajeeth T

1 October 2026

Table of Contents

Target cloud: AWS (ap-south-1) **CI/CD:** GitHub Actions **Reference service:** Inventory

Companion repository: see README .md for the code, Terraform, Helm chart, pipelines and DB scripts referenced throughout.

01 – Migration Assessment & Strategy (Order Management Domain)

Target cloud: AWS (single account per environment, region **ap-south-1** primary, **ap-southeast-1** for DR snapshots). **Why AWS:** mature DMS with CDC for MySQL, EKS + IRSA for fine-grained pod IAM (needed for PCI isolation of the payment service), native Secrets Manager rotation for RDS, and S3 lifecycle tiers that fit the 2 TB NFS data. Azure would be equally viable; the design below is portable at the Kubernetes/Helm/pipeline layer and only the Terraform and DB-migration tooling are AWS specific.

All figures and the FinNova environment are fictional per the assessment brief. No employer data is used.

1. Assumptions

#	Assumption
A1	Site-to-site connectivity (AWS Site-to-Site VPN, upgradable to Direct Connect) exists between the VMware DC and AWS before DB migration begins.
A2	On-prem MySQL 5.7 has binary logging enabled (or can be enabled with a restart in a planned window before migration).
A3	Peak write rate on the primary is $\leq 2,000$ TPS; average binlog volume ≤ 40 GB/day.
A4	Monolith split is in progress: order, inventory and payment each own a logical schema inside the same MySQL instance today. They will share one RDS instance in phase 1 with per-service DB users, and split to separate instances later.
A5	PCI scope: payment service handles card tokens from a PSP (no raw PAN stored). Isolation is still required to reduce audit scope (PCI DSS segmentation).
A6	Static assets, user uploads and reports on NFS are ~ 2 TB; $\sim 70\%$ has not been read in 90 days.
A7	MySQL 5.7 is end-of-life; target is MySQL 8.0 (RDS). Application compatibility with 8.0 (auth plugin, reserved words, <code>sql_mode</code>) is tested in the staging replica before cutover.

2. 6R Classification

Component	Strategy	Justification
Web tier (Java/Node, 2 VMs + Nginx LB)	Replatform	Lift-and-shift VMs would not give blue/green or elastic scaling for sales events. Containerizing the existing app and running it on EKS behind an ALB (weighted target groups / Argo Rollouts) delivers canary and blue/green with minimal code change. Nginx LB is replaced by ALB + Ingress.
Order Processing – order service	Refactor (continue the in-flight split)	The monolith is already being decomposed; landing it directly as a containerized microservice avoids migrating a monolith twice. Stateless, scaled by HPA, DB credentials from Secrets Manager.
Order Processing – inventory service	Refactor	Same rationale; selected as the reference service in this PoC (Dockerfile, Helm, pipeline) because it is the lowest-risk to prove the pattern.
Order Processing – payment service	Refactor + isolate	Needs a dedicated node group, subnet, security group, IAM role and NetworkPolicy to shrink PCI scope. Cannot be a simple rehost because isolation and independent deploys are explicit requirements.
Primary DB (MySQL 5.7, 800 GB)	Replatform	Move to managed RDS MySQL 8.0 Multi-AZ with the same relational engine (no schema refactor), gaining automated backups, patching, and KMS encryption. DMS full load + CDC keeps the cutover under 5 minutes. Aurora was considered but rejected for PoC cost; revisit at 10x scale.
Batch / reporting (nightly cron → NFS)	Replatform	Convert cron to Kubernetes CronJobs on a Spot-backed node group writing to S3. Cost-efficient, low priority, tolerant to interruption (jobs are idempotent and re-runnable).
File storage (2 TB NFS)	Replatform	Move to S3 with lifecycle (Standard → Standard-IA at 30 days → Glacier Instant Retrieval at 90 days). Initial copy with AWS DataSync, then delta sync. Code that needs POSIX semantics uses the S3 Mountpoint CSI driver for read access during transition.
Auth & secrets (static files with DB creds)	Replatform (+ Retire the static config files)	Secrets move to AWS Secrets Manager (KMS CMK, rotation). Pods read them at runtime through the Secrets Store CSI driver and IRSA. The plaintext config files are retired and old credentials rotated at cutover.

No component is marked *Repurchase*; *Retain* is limited to the on-prem MySQL primary until cutover completes and a 14-day rollback window ends. Retire applies to NFS server, Nginx VMs and static config files after the window.

3. Dependency Map

(Mermaid source omitted in this export; editable source is in the repository markdown.)

From	To	Protocol / nature	Criticality
Customers	Nginx LB → ALB	HTTPS	High
Web tier	Order service	HTTP/gRPC (internal)	High
Order service	Inventory service	HTTP (reserve/release stock)	High
Order service	Payment service	HTTPS mTLS, idempotency key	Critical

From	To	Protocol / nature	Criticality
Order / inventory / payment	MySQL (own schema each)	TCP 3306	Critical
Payment service	External PSP	HTTPS	Critical
Web tier, batch	File storage (NFS → S3)	NFS / S3 API	Medium
Batch jobs	MySQL and file storage	TCP 3306, S3	Low
All services	Secrets store	Credentials at startup / rotation	Critical

Critical-path dependencies: **network + secrets → database → services → web tier → batch/files**.
The database is the single shared dependency, so it drives sequencing.

4. Risk Register

ID	Risk	Likelihood	Impact	Mitigation	Owner
R1	Data loss or divergence during DB cutover	Med	Critical	DMS CDC with validation enabled; freeze writes (read-only) before final check; compare checksums; keep on-prem primary untouched with reverse replication for rollback.	DB lead
R2	Downtime exceeds the 5-minute window	Med	Critical	Rehearse the cutover ≥ 3 times in staging with prod-sized data; scripted cutover (no manual steps); lower DNS TTL to 30 s a day earlier; pre-warm connection pools.	Migration lead
R3	Credential exposure (plaintext configs, secrets in images or Git)	High	Critical	Secrets Manager + CSI; <code>git leaks</code> in CI; Trivy secret scan on image; rotate all legacy credentials at cutover; no secrets in env vars or Helm values.	Security
R4	PCI scope creep (payment traffic mixed with other tiers)	Med	High	Dedicated subnet, node group, SG, IAM role, NetworkPolicy default-deny; VPC Flow Logs and GuardDuty; quarterly segmentation test.	Security
R5	Cost overrun (NAT data, over-provisioned DB, DMS left running)	High	Med	AWS Budgets with alerts at 50/80/100 %; right-size after 2 weeks of metrics; VPC endpoints to cut NAT traffic; DMS instance destroyed after validation; tag-based cost allocation.	FinOps

ID	Risk	Likelihood	Impact	Mitigation	Owner
R6	MySQL 5.7 – 8.0 incompatibility (collation, auth plugin, deprecated syntax)	Med	High	Run the app test-suite against an 8.0 replica in staging; <code>mysqlsh util.checkForServerUpgrade</code> ; keep <code>sql_mode</code> aligned.	App leads
R7	Replication lag or WAN bandwidth too low for the 800 GB full load	Med	High	Measure throughput ahead of time; use parallel DMS load (<code>MaxFullLoadSubTasks</code>), run full load off-peak; fall back to Snowball Edge if sustained < 100 Mbps.	Network
R8	Hidden dependencies on NFS paths or hard-coded IPs	Med	Med	Dependency discovery with Application Discovery Service + <code>ss/lsof</code> traces for two weeks; config audit; feature flag for file backend.	App leads
R9	Canary/blue-green release bug blocks sales event	Low	High	Change freeze 2 weeks before major sales; fast rollback via <code>helm rollback</code> ; load test at 2x last peak.	Release mgr
R10	State file loss or concurrent terraform apply	Low	High	S3 versioning + DynamoDB locking + per-env state; CI-only applies.	Platform

5. Phased Migration Sequence

Phase	Scope	Why this order	Exit criteria
0 – Foundations (week 1–2)	AWS landing zone, VPN, IaC pipeline, VPC, EKS (empty), Secrets Manager, ECR, observability	Everything else depends on network, identity and secrets. Low business risk.	Terraform applies cleanly in dev/staging; VPN tested.
1 – Data plane preparation (week 2–4)	S3 + DataSync for NFS (initial copy), RDS 8.0, DMS full load + CDC running, validation job	Longest lead-time activity (800 GB + 2 TB); running in parallel de-risks the cutover. Does not change production traffic.	CDC lag < 5 s steady; validation 100 % match.
2 – Stateless services & batch (week 4–6)	Inventory (pilot), then order, then batch CronJobs, pointing at the replica/RDS in staging	Low business priority first to prove pipeline, Helm, secrets; batch is low priority and cost-sensitive.	Staging soak test passes; pipeline promotes to prod-shadow.
3 – Payment service (week 6–7)	Dedicated PCI node group, NetworkPolicy, security review, canary	Critical + regulated; needs the pattern proven by phases 1–2.	Segmentation test and security sign-off.
4 – Web tier cutover + DB cutover (week 8)	Weighted DNS for web tier (10 – 50 – 100 %); DB cutover in a < 5 min window	Business-critical; done last when everything else is verified; DB cutover is combined with a low-traffic window.	Error rate and latency within SLO for 48 h.
5 – Decommission (week 10–12)	After 14-day rollback window: stop reverse replication, retire NFS, VMs, rotate legacy	Avoid paying twice; reduce attack surface.	Sign-off from business owner.

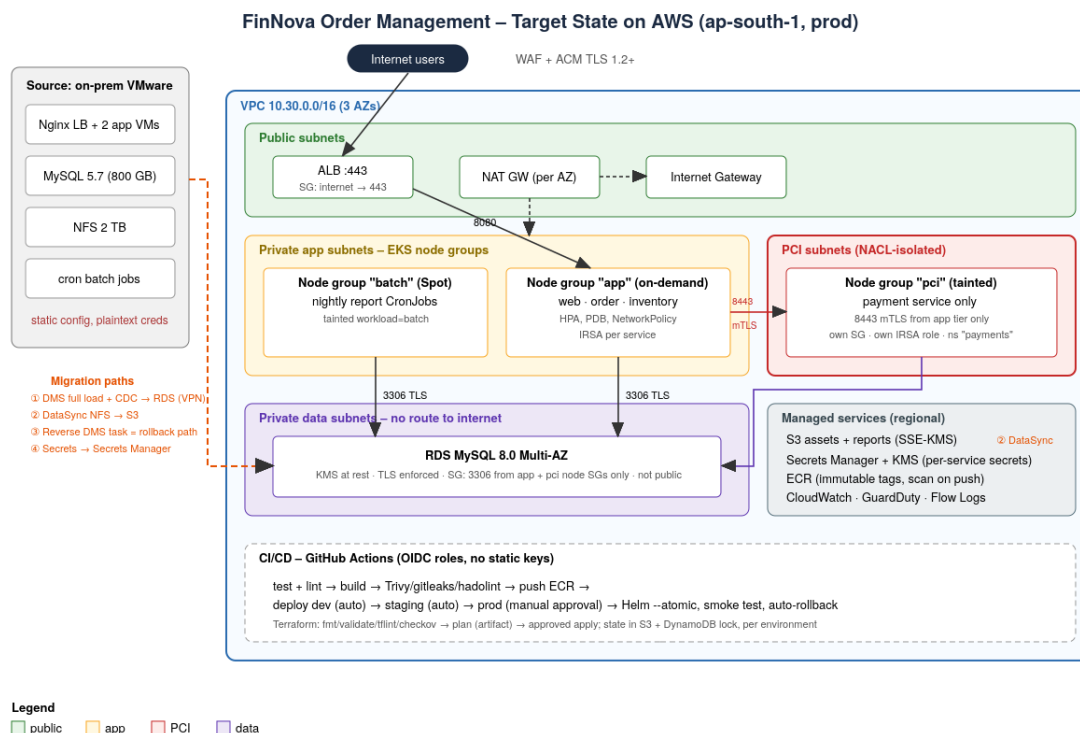
Phase	Scope	Why this order	Exit criteria
	credentials		

02 – Target Architecture & Infrastructure as Code

1. Source vs target

Layer	Source (on-prem VMware)	Target (AWS, ap-south-1)
Edge / LB	Nginx on VM	ALB (AWS Load Balancer Controller), TLS 1.2+ terminated with ACM, AWS WAF
Web / services	2 app VMs	Containers on EKS (private subnets), HPA + Cluster Autoscaler
Payment	Same VMs as monolith	Dedicated PCI node group + subnet + SG + IRSA role + NetworkPolicy
Database	MySQL 5.7 master/replica	RDS MySQL 8.0 Multi-AZ, private data subnets
Batch	cron on VM	Kubernetes CronJobs on Spot node group
Files	2 TB NFS	S3 (SSE-KMS, lifecycle tiers)
Secrets	Static config files	Secrets Manager + KMS, mounted by Secrets Store CSI via IRSA
CI/CD	Manual	GitHub Actions → ECR → Helm → EKS

2. Target-state diagram



Target-state architecture

The image version is docs/architecture.svg (also rendered into the Word/PDF pack). Editable Mermaid source:

(Mermaid source omitted in this export; editable source is in the repository markdown.)

3. Terraform layout

```

terraform/
├── bootstrap/                                # one-time: state bucket (KMS, versioned) +
DynamoDB lock table
├── modules/
│   ├── network/                            # VPC, 4 subnet tiers x AZ, IGW, NAT, route
tables, NACL, SGs, S3 endpoint, flow logs
│   ├── eks/                                # cluster, KMS secrets encryption, OIDC/IRSA,
3 node groups, add-ons
│   ├── rds/                                # MySQL 8.0, param group (TLS, ROW binlog),
KMS, managed master secret
│   ├── secrets/                            # Secrets Manager secrets + per-service
least-privilege IRSA roles
│   └── storage/                            # S3 assets/reports buckets + lifecycle
└── stack/                                  # ONE root module, instantiated per
environment
    ├── main.tf variables.tf outputs.tf versions.tf
    └── envs/
        ├── dev.tfvars      dev.backend.hcl
        ├── staging.tfvars  staging.backend.hcl
        └── prod.tfvars     prod.backend.hcl

```

Reusability across dev / staging / prod

- Modules contain **no environment logic**; everything differing between environments is a variable (single_nat_gateway, node sizes, db_multi_az, deletion_protection, retention).
- The root stack/ is identical for every environment; behavior is selected with -var-file=envs/<env>.tfvars and -backend-config=envs/<env>.backend.hcl.
- Workspaces were considered and **rejected** for environment separation: they share one backend/credentials and make it easy to apply to the wrong environment. Separate state per environment (and ideally per AWS account) gives hard isolation.

Setting	dev	staging	prod
AZs	2	2	3
NAT	1 shared	1 shared	1 per AZ
App nodes	t3.large Spot, 1-3	m6i.large, 2-4	m6i.xlarge, 3-12
PCI nodes	t3.large, 1-2	m6i.large, 2-3	m6i.large, 3-6
RDS	db.t4g.medium, single-AZ, 100 GB	db.r6g.large, single-AZ, 1 TB	db.r6g.xlarge, Multi-AZ, 1 TB
Backup retention	3 d	7 d	14 d
Deletion protection	off	on	on
EKS API endpoint	private	private	private

State management

Concern	Approach
Backend	S3 bucket per environment (finnova-tfstate-<env>-<account>), created by bootstrap/
Locking	DynamoDB table finnova-tf-locks (LockID), prevents concurrent applies
Encryption	KMS CMK (alias/finnova-tfstate), TLS only, public access blocked
Versioning / recovery	S3 versioning on; DynamoDB PITR on; prevent_destroy on the

Concern	Approach
	bucket
Isolation	Separate bucket, key and (recommended) separate AWS account per environment; CI roles are per-env
Access	CI only. Humans have read-only; apply happens from the reviewed plan artifact (<code>terraform.yml</code>)
Secrets in state	Avoided: RDS uses <code>manage_master_user_password</code> (secret lives in Secrets Manager); app secrets are placeholders with <code>ignore_changes</code>

4. Security-by-default (called out explicitly)

Control	Implementation
No public DB	<code>publicly_accessible = false</code> , data subnets have no default route , DB SG allows 3306 only from the app and PCI node SGs (SG references, no CIDRs)
Encryption at rest	RDS (KMS CMK, also PI + logs), S3 SSE-KMS with bucket keys, EBS node volumes encrypted, EKS secrets envelope-encrypted with KMS, ECR KMS, Terraform state KMS
Encryption in transit	RDS <code>require_secure_transport=1</code> ; S3 bucket policy denies <code>aws:SecureTransport=false</code> ; ALB TLS 1.2+; service-to-payment mTLS on 8443; DMS endpoints use SSL; IMDSv2-only on nodes
Least-privilege IAM	One IRSA role per service; policy = <code>GetSecretValue</code> on its own secret ARN + <code>kms:Decrypt</code> via Secrets Manager only; node roles carry only the three AWS-managed minimum policies; CI roles split build (ECR push), deploy (EKS namespace) and Terraform plan (read-only) vs apply
Network	Default SG locked; SG-to-SG rules; NACL on PCI subnets; EKS API private; VPC flow logs; S3 gateway endpoint
Supply chain	Immutable ECR tags, scan on push, Trivy + gitleaks + hadolint in CI, distroless non-root image
Kubernetes	Pod <code>runAsNonRoot</code> , read-only root FS, all caps dropped, seccomp RuntimeDefault, NetworkPolicies default-deny

5. Assumptions & intentional gaps (stated per the ground rules)

- Code is validated (`terraform validate`, `helm lint`) but **not applied** to a live account in this submission; costs would not be justified for a PoC. Expected behaviour is described in the runbook.
- Not included (listed for the follow-up discussion): AWS Load Balancer Controller / Secrets Store CSI / ASCP / Cluster Autoscaler (or Karpenter) / Argo Rollouts installs (Helm releases in a `platform/` stack), WAF rules, GuardDuty/Security Hub, Direct Connect, DR region.
- At 10x scale the first things to break are: single-writer RDS (move to Aurora + read replicas, split schemas per service), NAT data charges (add interface endpoints / VPC-native egress), and CoreDNS/ALB target registration limits.

03 – Database Migration Plan (MySQL 5.7, 800 GB → RDS MySQL 8.0)

1. Options considered

Option	Downtime	Verdict
<code>mysqldump</code> / <code>mydumper</code> + restore	Hours (800 GB)	Rejected – blows the 5 min budget
Percona XtraBackup → S3 → RDS	Needs binlog catch-up; RDS MySQL only restores 5.6/5.7 (not 8.0) and we want 8.0	Rejected

Option	Downtime	Verdict
Native binlog replication (RDS as replica)	Minutes	Viable, but RDS cannot be an external replica without <code>mysql.rds_set_external_master</code> tuning, and the 5.7→8.0 hop plus reverse path is harder to operate
Dual-write in the application	Near zero	Rejected – requires code change in code being split, high consistency risk
AWS DMS full-load + CDC	Write freeze ≈ 2–4 min	Chosen – supports 5.7→8.0, built-in validation, restartable, and a reverse task gives a clean rollback

2. Design

```

On-prem MySQL 5.7 (read replica as DMS source)
  | Site-to-Site VPN (TLS)
  ▼
DMS replication instance (dms.r5.xlarge, Multi-AZ, private)
  | Phase A: parallel full load (8 threads, large tables range-
partitioned)
  | Phase B: CDC from the captured binlog position (cached changes
applied)
  ▼
RDS MySQL 8.0 Multi-AZ (private data subnet, TLS-only)

```

Key decisions:

- **Read from the on-prem replica**, not the primary, to avoid impacting production; replica needs `log_slave_updates=ON`.
- **Schema first** with `mysqldump --no-data` (tables, indexes, FKs, routines, triggers) because DMS does not migrate secondary objects well; DMS runs with `TargetTablePrepMode=DO_NOTHING` and FK checks disabled during load.
- **Upgrade 5.7 → 8.0 on the way**: collation normalised to `utf8mb4_0900_ai_ci`, `mysqlsh util.checkForServerUpgrade` run first, staging app suite executed against the RDS 8.0 target.
- **Per-service DB users** are created on RDS at cutover; legacy credentials are never copied.
- Pre-flight: tables with no primary key must get one (query in `db/00-source-prereqs.sql`).
- Bandwidth: 800 GB at a sustained 200 Mbps ≈ 9 h. Full load runs over a weekend; if the VPN cannot sustain ≥ 100 Mbps use AWS Snowball Edge for the initial load and DMS CDC for catch-up.

3. Full-load step (commands)

Implemented in `db/01-schema-and-full-load.sh`; the essential commands:

```

# 1. schema only
mysqldump -h $SRC_HOST -u $SRC_USER -p"$SRC_PASS" --ssl-mode=REQUIRED \
  --no-data --routines --triggers --events --single-transaction \
  --databases orders inventory payment > schema.sql
mysql -h $TGT_HOST -u $TGT_USER -p"$TGT_PASS" --ssl-mode=VERIFY_IDENTITY <
schema.sql

```

2. DMS instance + endpoints

```
aws dms create-replication-instance --replication-instance-identifier
finnova-dms-ri \
  --replication-instance-class dms.r5.xlarge --allocated-storage 300 --
multi-az --no-publicly-accessible \
  --vpc-security-group-ids $DMS_SG_ID --replication-subnet-group-identifier
finnova-dms
aws dms create-endpoint --endpoint-identifier src-onprem-mysql57 --
endpoint-type source --engine-name mysql ...
aws dms create-endpoint --endpoint-identifier tgt-rds-mysql80 --endpoint-
type target --engine-name mysql ...
```

3. task: full load + ongoing replication

```
aws dms create-replication-task --replication-task-identifier finnova-
orders-full-cdc \
  --migration-type full-load-and-cdc \
  --table-mappings file://table-mappings.json \
  --replication-task-settings file://task-settings.json ...
aws dms start-replication-task --replication-task-arn $TASK_ARN --start-
replication-task-type start-replication
```

Config files: db/table-mappings.json (three schemas, temp tables excluded, order_items range-partitioned for parallel load) and db/task-settings.json (batch apply, ValidationSettings.EnableValidation=true, 8 parallel full-load subtasks, LOB limited mode).

Password handling: credentials are fetched from Secrets Manager inside the script and never written to disk or Git.

4. Cutover (target < 5 minutes of write freeze)

Scripted in db/03-cutover.sh so that nothing is typed during the window.

T+	Action	Control
T-1 day	Lower DNS TTL to 30 s; confirm CDC lag < 5 s for 24 h; run full 02-validate.sh	Go/No-Go checklist
T-30 min	Pre-scale AWS pods, warm connection pools, announce	
0:00	Maintenance page on, scale writers to 0, set source super_read_only=ON, kill stragglers	Writes stopped
0:30	Wait until DMS CDCLatencyTarget ≤ 1 s	Source and target converge
1:30	Fast validation (counts, invariants, sampled checksums)	Fail ⇒ rollback #1
2:00	Stop forward task, start reverse task (RDS – on-prem) , restore innodb_flush_log_at_trx_commit=1	Safety net
2:15	Create service users + write new Secrets Manager values, scale AWS pods up	New credentials only
3:30	Smoke tests incl. canary write/read	Fail ⇒ rollback #2
4:00	Lift maintenance page / flip DNS to the ALB	Cutover done

Rehearsal: at least three full dress rehearsals in staging against a production-sized restore; time each step and tighten the script until p95 < 4 min.

5. Validation approach

Four layers, from cheap to thorough (db/02-validate.sh):

1. **DMS row-level validation** – continuous during CDC; any Mismatched records fails the gate.
2. **Exact row counts** for every table on both sides.
3. **Chunked checksums** – each table is split into PK ranges of 200 k rows; COUNT (*) plus a BIT_XOR of per-row MD5 is compared per range. Catches silent value drift that counts miss. Full pass before cutover, a 2 % sample inside the freeze window to stay inside the time budget, 100 % again after cutover (in the background, read-only).
4. **Business invariants** – order count/total/max id, stock-on-hand sums, payment transaction totals.

Post-cutover monitoring for 48 h: replication to on-prem lag, error rate, p95 latency, deadlocks, slow-query log, RDS CPU/IOPS/connections.

6. Rollback plan

Situation	Action	Data loss
Validation fails inside the freeze (apps still stopped)	04-rollback.sh --before-switch: remove read_only, lift maintenance page. On-prem was never replaced.	None
Smoke test fails or severe issue within the first hours (writes already on RDS)	04-rollback.sh --after-switch: stop AWS writers, let the reverse DMS task drain RDS – on-prem, stop it, enable on-prem writes, repoint traffic.	None (reverse CDC kept on-prem current)
Problem found days later	Same procedure while the reverse task is still running (window: 14 days). After that, point-in-time restore of RDS + forward fix.	Bounded by RPO of the restore
Reverse replication broken	Fall back to the last RDS snapshot taken at T+0 and the on-prem binlogs; escalate (documented as the residual risk).	Writes made after cutover since the break

On-prem databases and the DMS instance are kept for the 14-day window, then decommissioned after sign-off.

7. Post-load tuning

- After full load: set innodb_flush_log_at_trx_commit=1 (durability), enable Performance Insights review, ANALYZE TABLE on large tables, confirm optimizer plans for the top 20 queries on 8.0.
- Add RDS Proxy later (connection pooling for HPA bursts) – deferred to keep the PoC small.

04 – Network Segmentation, Security & Secrets

1. Network segmentation

CIDR plan (prod 10.30.0.0/16, one /20 per subnet per AZ):

Tier	Subnets	Contains	Route to internet	Ingress allowed from
Public	10.30.0.0/20 ...	ALB, NAT gateways	IGW	Internet → 443 (ALB SG only)
App	10.30.64.0/20 ...	EKS app nodes (web, order, inventory), batch Spot nodes	Egress via NAT	ALB SG → 8080
PCI	10.30.128.0/20 ...	EKS pci nodes (payment service only)	Egress via NAT, HTTPS only	App SG → 8443 only
Data	10.30.192.0/20 ...	RDS	None	App SG + PCI SG → 3306

Allowed flows (everything else is denied)

From	To	Port	Enforced by
Internet	ALB	443	ALB SG
ALB	web/order pods	8080	App-node SG referencing ALB SG
order service	payment service	8443 (mTLS)	PCI-node SG (source = App SG), PCI NACL, Kubernetes NetworkPolicy
order / inventory / batch	RDS	3306	DB SG (source = App SG)
payment	RDS	3306	DB SG (source = PCI SG)
payment	PSP / AWS APIs	443	PCI SG egress, NACL egress 443
pods	Secrets Manager / STS	443	via NAT (or interface endpoints)
anything else, incl. App ⇌ PCI on other ports, PCI → internet on other ports, Internet → data			denied

How the payment service is isolated

Defence in depth: five independent layers, so a mistake in one does not break isolation.

1. **Subnet + route table:** dedicated PCI subnets and route tables, no peering with other tiers other than the allowed flows.
2. **Network ACL** (stateless): inbound 8443 only from app subnets; outbound 443 (PSP), 3306 to data subnets, ephemeral replies. Everything else is implicitly denied.
3. **Security group:** pci - nodes SG accepts only 8443 from app - nodes SG. Not shared with any other workload.
4. **Dedicated node group:** nodes labelled `pci=true` and tainted `pci=true:NoSchedule`. Only the payment Deployment tolerates the taint and has the matching `nodeSelector`, so no non-PCI pod shares a host.
5. **Identity & Kubernetes:** separate `payments` namespace; own `ServiceAccount` bound to its own IRSA role (`finnova-prod-payment-irsa`) that can read only the payment secret; default-deny `NetworkPolicy` allowing ingress only from the `orders` namespace pods labelled `app=order`; Pod Security Standard restricted on the namespace.

Plus: VPC Flow Logs on all traffic, EKS audit logs, GuardDuty (EKS + RDS protection) and a quarterly segmentation test (attempt blocked flows from app tier and verify they are denied and alerted) as PCI evidence.

2. Secrets: from static files to Secrets Manager

Before

Static config files on the VMs with embedded plaintext DB credentials (`db.properties/.env`), copied between servers, readable by anyone with shell access, not rotated.

After

Terraform —creates—> Secrets Manager secret `finnova-prod/inventory/db`
(KMS CMK, value set out-of-band)

`05-rotate-service-secrets.sh` ———▲ writes `{username,password,host,port}` at
cutover (never in Git/state)

| `GetSecretValue` (own secret only, via
IRSA)

Pod — `ServiceAccount (IRSA)` —> `Secrets Store CSI driver + AWS provider`
| mounts as a read-only file

`/mnt/secrets/db`

App reads `DB_SECRET_FILE` —> `loadDbConfig()` (re-read every 60 s =>
rotation without redeploy)

- **Storage:** one secret per service per environment, encrypted with a customer-managed KMS key. RDS admin credential is **RDS-managed** (`manage_master_user_password`), auto-rotated, and only DBAs/DMS roles can read it.
- **Retrieval at runtime:** the Helm chart creates a `SecretProviderClass` and mounts it with the CSI volume; pod identity comes from IRSA, so there are no AWS keys in the pod. The application reads the file path in `DB_SECRET_FILE` (see `services/inventory/src/config.js`).
- **No plaintext anywhere:** not in the image (Trivy secret scan + gitleaks in CI), not in environment variables, not in Helm values, not in Terraform state (placeholder with `ignore_changes`).
- **Rotation:** Secrets Manager rotation Lambda (MySQL single-user/alternating-user strategy) every 30 days; because the app re-reads the mounted file, rotation needs no restart. Legacy credentials from the old config files are revoked after the 14-day rollback window.

Least-privilege IAM (excerpt from `terraform/modules/secrets/main.tf`)

```
statement {
  sid      = "ReadOwnSecretOnly"
  actions  = ["secretsmanager:GetSecretValue",
"secretsmanager:DescribeSecret"]
  resources = [aws_secretsmanager_secret.svc[each.key].arn]
}
statement {
  sid      = "DecryptWithSecretsKeyOnly"
  actions  = ["kms:Decrypt"]
  resources = [aws_kms_key.secrets.arn]
  condition {
    test      = "StringEquals"
    variable  = "kms:ViaService"
    values    = ["secretsmanager.ap-south-1.amazonaws.com"]
  }
}
```

```
}
}
```

Trust policy: `sts:AssumeRoleWithWebIdentity` restricted to `system:serviceaccount:<namespace>:<service>` and `aud=sts.amazonaws.com`. The payment role is bound to the payments namespace only.

3. Other IAM / access boundaries

Principal	Allowed	Not allowed
GitHub Actions build role (OIDC, repo+branch condition)	ECR push to finnova/*	EKS, RDS, secrets
GitHub Actions deploy role (per env)	<code>eks:DescribeCluster</code> ; mapped through EKS access entry to a namespaced Kubernetes role	cluster-admin, other namespaces
Terraform plan role	Read-only	Any write
Terraform apply role	Provisioning, behind approval	Used from laptops (blocked by trust policy)
Node roles	EKS worker, ECR read-only, SSM core	Secrets Manager, S3, RDS
Humans	SSO, read-only by default; break-glass role with MFA and alerting	Static IAM users/keys

4. Encryption summary

Data	At rest	In transit
RDS	KMS CMK (storage, PI, logs)	TLS enforced (<code>require_secure_transport</code>), <code>VERIFY_IDENTITY</code> from clients
S3	SSE-KMS + bucket keys	Bucket policy denies non-TLS
EKS secrets / etcd	KMS envelope encryption	TLS to private API endpoint
EBS (nodes)	Encrypted gp3	–
Service ↔ payment	–	mTLS on 8443
DMS	Encrypted instance storage	SSL on both endpoints, over VPN
Terraform state	KMS-encrypted S3	TLS

05 – Runbook: Execution, Rollback, Validation

Audience: migration lead, DBA, platform engineer, release manager. Every phase has an owner, a go/no-go gate and a rollback.

0. Roles and communication

Role	Responsibility
Migration lead	Owns go/no-go, runs the bridge call
DBA	DMS, validation, cutover script
Platform engineer	Terraform, EKS, pipelines
App lead(s)	Smoke tests, app config
Security	Segmentation test, secrets sign-off
Business owner	Final sign-off, decommission approval

Comms: dedicated channel, status every 15 min during a cutover window, one rollback decider (migration lead).

1. Phase-by-phase execution

Phase 0 – Foundations (dev → staging → prod)

1. `cd terraform/bootstrap && terraform apply -var env=<env>` (once per account).
2. `cd terraform/stack && terraform init -backend-config=envs/<env>.backend.hcl && terraform plan -var-file=envs/<env>.tfvars -out tfplan --review -- terraform apply tfplan` (via the terraform workflow).
3. Install platform add-ons (Helm): AWS Load Balancer Controller, Secrets Store CSI driver + AWS provider, metrics-server, Cluster Autoscaler or Karpenter, Argo Rollouts.
4. Create GitHub Environments dev, staging, prod with variables (AWS_ROLE_ARN_DEPLOY, EKS_CLUSTER, ECR_REGISTRY, INVENTORY_IRSA_ROLE_ARN); add **required reviewers** to prod.
5. Establish the Site-to-Site VPN; test throughput with `iperf3` (record Mbps for DMS planning).

Gate: VPN stable 48 h, `terraform plan` shows no drift, node groups Ready, a hello-world pod can read a test secret via CSI.

Phase 1 – Data plane preparation

1. On-prem: run `db/00-source-prereqs.sql`; fix tables without primary keys; confirm binlog settings.
2. Start `db/01-schema-and-full-load.sh` (full load + CDC). Monitor `describe-table-statistics` and CloudWatch `CDCLatencyTarget`.
3. Create the reverse task with `db/01b-create-reverse-task.sh` (do not start).
4. NFS: `aws datasync create-task NFS -- finnova-prod-assets`; run the initial copy; schedule hourly delta syncs; verify with `--verify-mode ONLY_FILES_TRANSFERRED` then a final full verify.

Gate: CDC lag < 5 s for 24 h, `02-validate.sh` passes, DataSync delta < 5 min.

Phase 2 – Stateless services and batch (staging first)

1. Merge to main → pipeline builds, scans, pushes inventory and deploys to dev, then staging automatically.
2. Repeat for order. Convert nightly cron scripts to Kubernetes CronJobs on the batch Spot group writing to the reports bucket.
3. Run the functional + load test suite in staging against the RDS 8.0 replica.

Gate: staging soak 72 h without SLO breach; HPA scaling verified at 2x peak.

Phase 3 – Payment service

1. Deploy to the payments namespace on the PCI nodes with the canary procedure in section 3.
2. Run the segmentation test (section 5.4) and obtain Security sign-off.

Gate: all negative network tests blocked, secret access limited to the payment secret.

Phase 4 – Cutover (change window, low-traffic period)

Pre-flight checklist (all must be ✓):

- ☐ CDC lag < 5 s for 24 h; full 02-validate.sh passed within last 24 h
- ☐ Three successful staging rehearsals with timing recorded (p95 < 4 min)
- ☐ DNS TTL lowered to 30 s a day earlier
- ☐ Maintenance page tested; on-prem Ansible playbooks verified
- ☐ Reverse task created and test-started once in staging
- ☐ Rollback decider and DBA present; on-call engineers acknowledged
- ☐ Change approved; no sales event in the next 72 h

Execution:

1. export TASK_ARN=... REVERSE_TASK_ARN=... SRC_HOST=... TGT_HOST=...
EKS_CONTEXT=... ENV_NAME=prod
2. ./db/03-cutover.sh (scripted, timings printed). Watch the output on the bridge.
3. Shift web traffic: Route 53 weighted records 10 % → 50 % → 100 % to the ALB, 10 min at each step (the web tier can be moved before or after the DB depending on rehearsal results; the DB cutover is the single atomic step).

Phase 5 – Hypercare and decommission

- 48 h hypercare with tightened alerts; daily 02-validate.sh in read-only mode against on-prem (reverse task) for 14 days.
- After sign-off: stop the reverse task, delete DMS instance, take final on-prem snapshots, power off VMs, delete NFS after 30 more days, revoke legacy credentials, remove VPN if no longer needed.

2. Rollback matrix

Trigger	Decision window	Procedure	Owner
Terraform apply fails	Any time	Fix forward or terraform apply the previous commit; state is versioned in S3	Platform
Helm deployment fails readiness	Automatic	helm upgrade --atomic rolls back on its own	Pipeline
Smoke test fails after Helm success	Automatic	_deploy.yml runs helm rollback	Pipeline
Bad release found later	< 5 min	inventory-service workflow dispatch with rollback_environment=prod (or helm rollback <release> <rev>)	Release mgr
DB validation fails in freeze	Immediate	./db/04-rollback.sh --before-switch	DBA
DB / app failure after cutover	14 days	./db/04-rollback.sh --after-switch (reverse replication already current)	DBA + lead
Web tier regression after traffic shift	Immediate	Set Route 53 weight back to 100 % on-prem (TTL 30 s)	Platform
Secret compromise	Immediate	Rotate the secret in Secrets Manager (CSI picks up within 60 s) and revoke the IRSA role session	Security

3. Zero-downtime deployment for the payment service (PCI-aware)

Chosen strategy: canary with automated analysis (Argo Rollouts) on a dedicated PCI node group, with blue/green semantics for the database and message contracts.

Why not plain rolling update: payment requests are not safely retryable by clients and each release changes money-moving code, so we want a *measured* exposure (1 % → 10 % → 50 % → 100 %) with automatic abort, not instant replacement of all pods.

Why not full blue/green: a full duplicate environment would double the PCI-scoped footprint (more in-scope nodes, more audit evidence). A canary inside the existing PCI node group keeps the scope constant.

Mechanics:

1. **Capacity:** PCI node group has headroom (maxSurge=25 %, Cluster Autoscaler may add a node, always inside the PCI subnets with the same SG/IRSA). The canary pods use the same ServiceAccount, taint toleration and NetworkPolicy, so isolation is identical for old and new versions.
2. **Traffic split:** the order service calls the payment Service; Argo Rollouts shifts weight at the ALB target-group level (or via the Gateway API/service mesh) 1 % → 10 % → 50 % → 100 % with a pause and AnalysisTemplate between steps.
3. **Automated analysis** (Prometheus/CloudWatch): HTTP 5xx rate < 0.5 %, p95 latency < 400 ms, payment authorisation success rate not lower than baseline – 0.5 %, PSP error ratio, DB error ratio. Any breach aborts and returns 100 % to the stable ReplicaSet automatically.
4. **Request safety:** all payment calls carry an **idempotency key**; the service persists it, so retries during pod termination cannot double-charge. In-flight requests finish: preStop sleep 15 s, terminationGracePeriodSeconds 60 s, readiness fails first on SIGTERM, ALB deregistration delay 30 s.
5. **Database changes:** expand/contract only. Release N adds nullable columns/tables (compatible with N-1), release N+1 starts using them, release N+2 removes old structures. This is what makes rollback of the app safe at any step.
6. **Availability guards:** PDB minAvailable: 2, topology spread across AZs, readiness probe that verifies PSP connectivity and DB pool, startup probe to avoid premature traffic.
7. **PCI compliance during deploy:** the image comes only from the hardened pipeline (scan gate, signed with cosign, admission policy verifies signature), deployment needs a **two-person approval** (GitHub Environment reviewers), every promotion is logged (change record = workflow run + commit SHA) and no human kubectl access to the namespace.
8. **Rollback:** automated on analysis failure; manual is `kubectl argo rollouts abort/undo payment -n payments` or `helm rollback`.

4. Post-migration validation checklist

Data

- ☐ `02-validate.sh` passes on RDS vs on-prem (all four layers)
- ☐ Auto-increment values on RDS ≥ source max + safety margin
- ☐ No tables without primary keys; FKs/triggers/routines present (`information_schema` diff)

- ☐ File counts and checksums match between NFS and S3 (DataSync verify report)

Application

- ☐ All pods Ready; zero restarts in 1 h; HPA reacts to a load test
- ☐ Order → inventory → payment end-to-end test order succeeds
- ☐ Nightly report job produced its report in S3 and in the same format
- ☐ Canary order cleaned up

Security

- ☐ No plaintext credentials in repos, images (Trivy), configs or env vars (`kubectl describe` check)
- ☐ RDS not publicly accessible; DB SG sources limited to the two node SGs
- ☐ Negative network tests blocked (section 5.4)
- ☐ CloudTrail/Config/GuardDuty enabled; flow logs flowing
- ☐ Legacy credentials revoked (after rollback window)

Operations

- ☐ Alarms: RDS CPU/connections/replica lag, 5xx, pod restarts, node pressure, NAT bytes, budget alerts
- ☐ Backups: RDS automated backup exists, one restore test performed
- ☐ Runbook and on-call rotation updated

Cost

- ☐ Budgets at 50 / 80 / 100 %, cost allocation tags visible, DMS instance removed, Savings Plan evaluated after 2 weeks of data

5. Failure drill: failed production cutover in real time (for the follow-up discussion)

1. **Detect:** validation fails, smoke test fails, error rate > 2 % or the 5-minute budget is about to be exceeded.
2. **Decide:** the migration lead decides within 60 s; bias to rollback if there is any doubt about data integrity.
3. **Act:** before the switch — `04-rollback.sh --before-switch` (seconds). After the switch — `--after-switch`: stop AWS writers, drain the reverse task, enable on-prem.
4. **Verify:** row counts + invariants in the *reverse* direction, smoke test against on-prem, lift the maintenance page.
5. **Communicate** and open a blameless review: capture timings, logs, root cause; schedule a new rehearsal before the next attempt.

5.4 Segmentation test (run in Phase 3 and quarterly)

From an order pod: should SUCCEED (allowed flow)

```
kubectl -n orders exec deploy/order -- wget -q0- --timeout=3
https://payment.payments.svc:8443/healthz
```

From inventory pod: should FAIL (NetworkPolicy)

```
kubectl -n orders exec deploy/inventory -- wget -q0- --timeout=3
https://payment.payments.svc:8443/healthz
```

From a payment pod: reaching the batch namespace or the internet on 80

should FAIL

```
kubectl -n payments exec deploy/payment -- wget -q0- --timeout=3  
http://example.com
```

Secrets: payment role must not read other secrets

```
kubectl -n payments exec deploy/payment -- aws secretsmanager get-secret-  
value --secret-id finnova-prod/order/db # AccessDenied
```

06 – Cost Estimate & Optimization

Method and caveats. AWS figures are rough monthly **on-demand list prices for us-east-1 / ap-south-1 order of magnitude**, built from the AWS Pricing Calculator line items below and rounded. Mumbai prices are typically 5–15 % higher; re-run the calculator with the final region before committing. The “before” column is an **assumed** on-prem TCO for the fictional FinNova (hardware amortised over 5 years, VMware licences, power/cooling/colo share, support contracts) and must be replaced with finance actuals. Scope = the Order Management domain only.

1. Before vs after (production, monthly, USD)

Component	Before (on-prem, assumed)	After (AWS, on-demand)	After (with commitments)	Calculator line items
Compute (web/order/inventory/payment/batch)	3,200 (2 app VMs, batch VM, Nginx, share of hosts + VMware)	760	580	EKS control plane \$73; 3× m6i.xlarge app (~\$420); 3× m6i.large PCI (~\$210); batch Spot ~\$30; ALB ~\$25
Database	2,800 (2 DB hosts, MySQL support, SAN share, backup)	1,040	790	RDS db.r6g.xlarge Multi-AZ ~\$760; 1 TB gp3 Multi-AZ ~\$230; backup overage ~\$50
Storage (2 TB files)	900 (NFS array amortisation + maintenance)	35	35	S3 2 TB with lifecycle (~\$20 Standard + \$12 IA/GIR + requests)
Network / egress	600 (WAN, firewall share)	215	215	3 NAT GWs ~\$100 + data ~\$60, VPN ~\$40, data transfer out ~\$15
Security & observability	400	120	120	KMS + Secrets Manager ~\$25, CloudWatch logs/metrics ~\$70, GuardDuty ~\$25
Datacenter share (power, space, ops)	1,600	0	0	–
Total	≈ 9,500	≈ 2,170	≈ 1,740	

One-time migration costs (not in monthly): DMS replication instance ~\$130/month for about 2 months, DataSync ~\$0.0125/GB × 2 TB ≈ \$26, VPN, parallel running of both environments for ~1 month (~\$2k), staging rehearsals.

Non-production environments (monthly, on-demand)

Environment	Estimate	Notes
dev	~\$350	Spot t3 nodes, single NAT, db.t4g.medium; scheduled scale-to-zero nights/weekends can cut ~45 %
staging	~\$850	m6i.large, single NAT, db.r6g.large single-AZ; torn down between rehearsals if cost

Environment	Estimate	Notes
		matters

2. Optimization opportunities

#	Opportunity	Estimated saving	How
1	Savings Plan / Reserved Instances for steady load	~\$430/mo ($\approx 20\%$)	1-yr Compute Savings Plan on the 6 baseline nodes ($\approx 28\%$ of ~\$630 $\approx$ \$180), 1-yr RDS Reserved Instance on the instance only ($\approx 33\%$ of ~\$760 $\approx$ \$250); storage is not discounted. Commit only after 2–4 weeks of real metrics
2	Right-size RDS and nodes	\$100–250/mo	Start at db.r6g.xlarge; review Performance Insights/CloudWatch after 2 weeks; step down to db.r6g.large if CPU < 30 % and memory free > 50 %. Same for nodes using requests vs actual usage (VPA recommendations)
3	Spot for batch and non-prod	~70 % off those nodes (~\$60 prod+dev)	Reports are idempotent CronJobs on the batch-spot group with multiple instance types; dev apps on Spot
4	S3 lifecycle (implemented)	~\$20/mo vs Standard, more as data ages	Standard → IA at 30 d → Glacier IR at 90 d; 365-day expiry for reports; abort incomplete multipart uploads
5	Cut NAT data charges	\$30–80/mo	S3 gateway endpoint (implemented); add interface endpoints for ECR, STS, Secrets Manager, CloudWatch Logs; one NAT in non-prod
6	Autoscaling thresholds	10–20 % of compute	HPA CPU target 65 %, scale-down stabilisation 300 s; Cluster Autoscaler/Karpenter consolidation; scale non-prod to zero outside business hours
7	Graviton	~15–20 % better price/performance	Build multi-arch images; move to m7g/c7g nodes and r6g/r7g RDS (RDS already Graviton here)
8	Governance	Prevents overruns	AWS Budgets 50/80/100 %, Cost Anomaly Detection, mandatory tags (Environment, CostCenter, Owner) via provider default_tags, monthly FinOps review
9	Decommission quickly	~\$130/mo + double-running	Remove DMS instance and on-prem assets as soon as the rollback window closes

3. TCO reasoning (for the 6R discussion)

- Savings come mostly from removing capacity sized for the biggest sales peak: on-prem runs that capacity all year, AWS runs the baseline and bursts with HPA/autoscaling.
- The database is the largest AWS cost; Multi-AZ doubles it but replaces the on-prem replica and manual failover, so it is a like-for-like HA comparison.

- Break-even: migration effort ($\approx 2 \text{ engineers} \times 3 \text{ months} + \text{one-time costs}$) pays back in about 6–9 months at the estimated delta of $\sim \$7\text{k}/\text{month}$; sensitivity: if the true on-prem TCO is only $\$6\text{k}/\text{month}$ the payback moves to ~ 2 years and the argument rests on elasticity and resilience, not cost.

4. Reproducing the estimate

AWS Pricing Calculator (<https://calculator.aws>): create an estimate named `finnova-prod` with: Amazon EKS (1 cluster), EC2 ($3 \times \text{m6i.xlarge} + 3 \times \text{m6i.large}$, Linux, On-Demand, EBS 50 GB gp3 each), RDS for MySQL (`db.r6g.xlarge`, Multi-AZ, 1000 GB gp3, 12,000 IOPS/500 MB/s, 14 d backup), S3 (2,048 GB split Standard/IA/GIR), Application Load Balancer, NAT Gateway $\times 3$ (+ 2 TB/month processed), Secrets Manager (6 secrets), KMS (6 keys), CloudWatch (logs 100 GB/month), Site-to-Site VPN. Export the estimate as CSV and attach it with the submission.